

VITyarthi

Python Essentials

Personal Expense Management System

Project Report

Build Your Own Project

Name	Ashwin Ganeshrao Deshmukh
-------------	----------------------------------

Registration Number	26MIP10052
----------------------------	-------------------

Course	Python Essentials
---------------	-------------------

Submission Date	30 September 2026
------------------------	-------------------

Table of Contents

Table of Contents	1
1. Introduction	2
1.1 Objectives	2
2. Problem Statement	3
2.1 Scope	3
2.2 Target Users	3
3. Functional Requirements	3
4. Non-functional Requirements	4
5. System Architecture	5
6. Design Diagrams	6
6.1 Use Case Diagram	6
6.2 Workflow Diagram	7
6.3 Sequence Diagram	8
6.4 Class / Component Diagram	9
6.5 ER Diagram and Storage Design	9
7. Design Decisions and Rationale	10
8. Implementation Details	11
8.1 Amount validation	11
8.2 Main program	12
8.3 Analysis logic	12
9. Screenshots / Results	13
10. Testing Approach	16
11. Challenges Faced	16
12. Learnings and Key Takeaways	17
13. Future Enhancements	17
14. References	17

1. Introduction

Managing personal money is a skill that most students learn late. Income arrives from different sources such as pocket money, a part-time job or a salary, and it is spent on food, travel, shopping and bills, usually without any record. The Personal Expense Management System is a small Python program that helps a user record this information and understand it.

The system is a menu-driven command-line application. The user can add income, add expenses under categories, view every entry made so far and get an analysis of their finances that shows total income, total expense, balance, an approximate savings rate and the spending in each category. The project is written using only the concepts taught in the Python Essentials course (variables, operators, input and output, type conversion, data structures, control flow, functions and modules) and needs no external libraries.

1.1 Objectives

- To let the user record income and expenses quickly from the command line.
- To organise expenses by category so that the user can see where the money goes.
- To calculate the balance and savings rate automatically and give a clear status message.
- To validate every input so that wrong values do not break the program.
- To apply the Python Essentials concepts in a real-world problem using a modular file structure.
- To document, test and maintain the project using Git and GitHub.

2. Problem Statement

Many students and beginners do not keep a record of their money. By the end of the month they cannot say how much they earned, how much they spent, how much is left or which category took most of their money. Existing budgeting applications are often too complex, need internet access or an account, and are not suitable for someone who only wants a quick, simple summary.

There is a need for a small, easy-to-use tool that runs on the user's own computer, records income and expenses, and shows a clear picture of the user's finances.

2.1 Scope

Included: a Python 3 command-line program; recording income with a source; recording expenses with a category; viewing all entries; financial analysis; validation of amounts and menu choices.

Not included: permanent storage of data (entries are kept in memory), editing or deleting entries, a graphical or web interface, multiple users or login, and currencies other than the rupee.

2.2 Target Users

- Students who want to track pocket money and daily spending.
- Beginners and first-time earners who want a simple view of where their money goes.
- Anyone who prefers a small offline tool over a full budgeting application.

3. Functional Requirements

The functional requirements describe what the system must do. The system provides four major functional modules (Add Income, Add Expense, View History and Analysis) controlled by a main menu.

ID	Requirement	Module
FR1	The system shall display a main menu with the options Add Income, Add Expense, View History, Show Analysis and Exit, and shall show it again after every action.	main.py
FR2	The system shall let the user add an income entry consisting of a source and an amount. An empty source shall be stored as "Other".	add_income.py
FR3	The system shall let the user add an expense entry consisting of a category chosen from a list and an amount. An invalid category number shall be stored as "Other".	add_expense.py
FR4	The system shall accept an amount only if it is a positive number (digits with at most one decimal point) and shall ask again otherwise.	add_income.py, add_expense.py
FR5	The system shall display all income and expense entries added so far, or a message if there are none.	view_history.py
FR6	The system shall calculate and display total income, total expense and balance.	analysis.py
FR7	The system shall display the approximate savings rate and the expense total and percentage for each category.	analysis.py

ID	Requirement	Module
FR8	The system shall display a status message stating whether the user saved money, broke even or spent more than the income.	analysis.py
FR9	The system shall report an invalid menu choice and continue running until the user chooses Exit.	main.py

Input / output structure. Input is taken through the keyboard using input(): a menu number, an income source or category, and an amount. Output is printed on the terminal as confirmation messages, the transaction history and the finance analysis. **Workflow.** The user starts the program, sees the menu, chooses an option, completes the action and returns to the menu, repeating until Exit is chosen (see the workflow diagram in Section 7.2).

4. Non-functional Requirements

The following non-functional requirements define how well the system must work.

ID	Category	Requirement	How it is met
NFR1	Usability	The program shall be easy to use by a beginner without training.	Simple numbered menu, clear prompts (for example "Enter your income: ₹") and confirmation messages after every action.
NFR2	Reliability / Error handling	The program shall not crash because of wrong input.	Amounts are checked character by character before conversion; invalid amounts and menu choices show an error message and the program asks again.
NFR3	Maintainability / Modularity	The code shall be easy to understand and change.	One file per function, short functions, comments in every file and simple variable names. Changing one feature does not affect the others.
NFR4	Performance	Every action shall respond immediately.	All data is held in memory in lists and dictionaries; each action processes only a small number of entries, so response is instant.
NFR5	Portability	The program shall run on any system with Python 3.	Uses only standard Python features with no external libraries and no operating-system-specific code.
NFR6	Resource efficiency	The program shall use very little memory and no disk or network.	Only two lists of small dictionaries are stored; nothing is written to disk and no internet is needed.

5. System Architecture

The system follows a simple layered, modular architecture. The user interacts with the terminal. **main.py** acts as the controller: it owns the two data lists (incomes and expenses), shows the menu in a loop and decides which function to call. Each feature is written as a function in its own module file: `add_income.py`, `add_expense.py`, `view_history.py` and `analysis.py`. The data is stored in memory as lists of dictionaries.

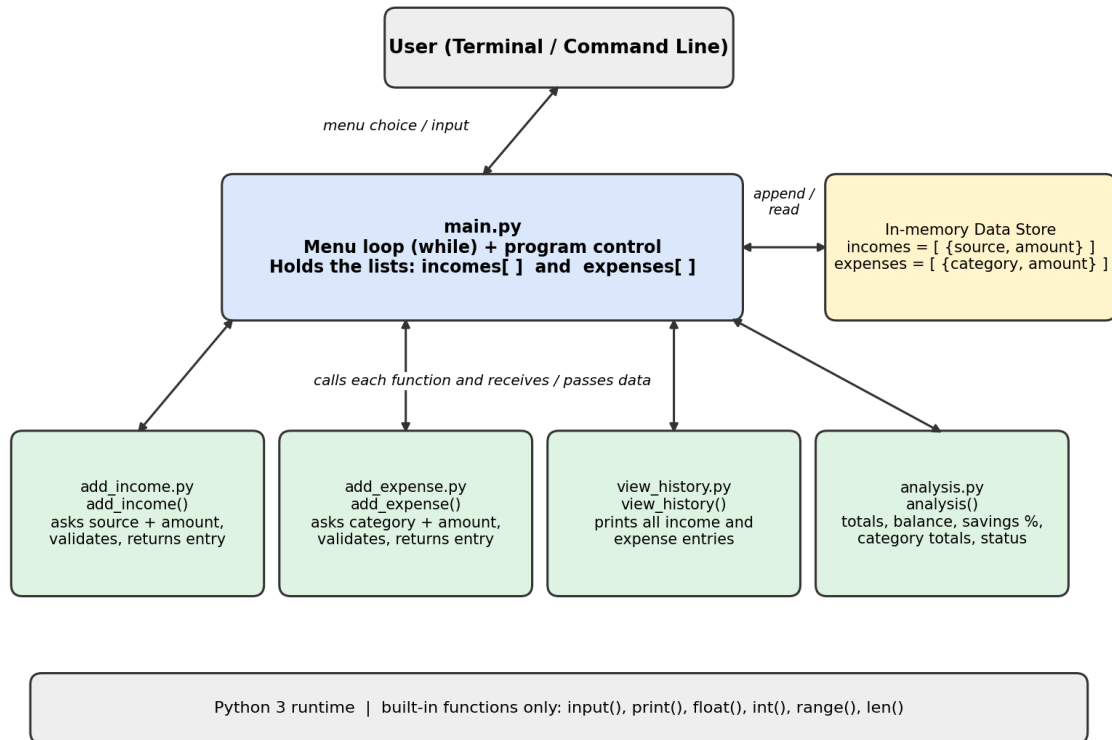


Figure 1: System architecture diagram

Data flow. The add functions read input from the user and return a dictionary; `main.py` appends it to the correct list. The `view_history` and `analysis` functions receive both lists as parameters and only read from them, so they never change the stored data.

6. Design Diagrams

6.1 Use Case Diagram

The only actor is the User. The system offers five use cases. Add Income and Add Expense both include the Validate Amount use case, because both check the amount before saving it.

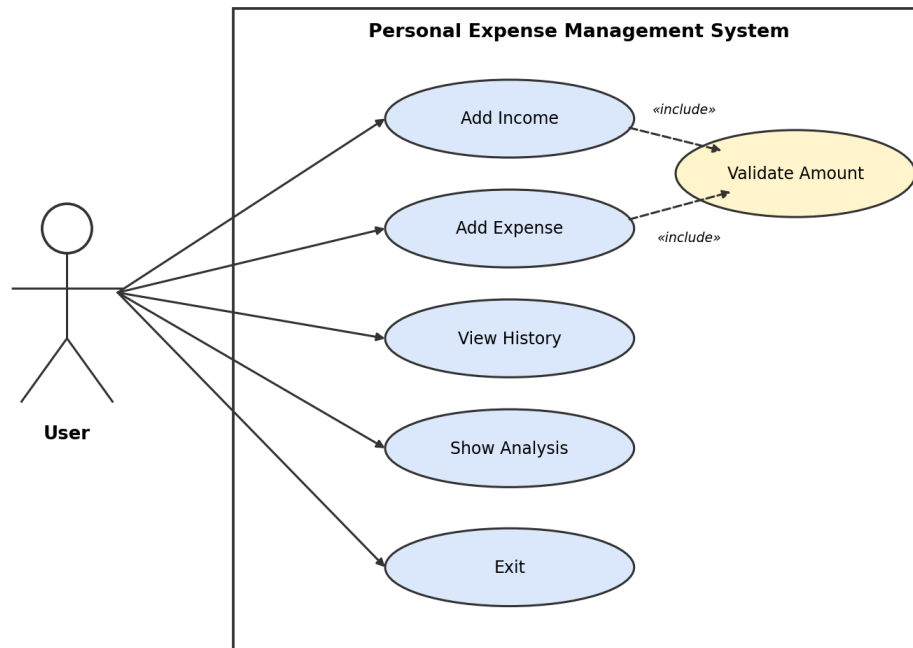


Figure 2: Use case diagram

6.2 Workflow Diagram

The workflow shows how the program runs. The menu is displayed, the choice is read and one of the branches is executed. Every branch except Exit returns to the menu, so the program continues until the user chooses option 5.

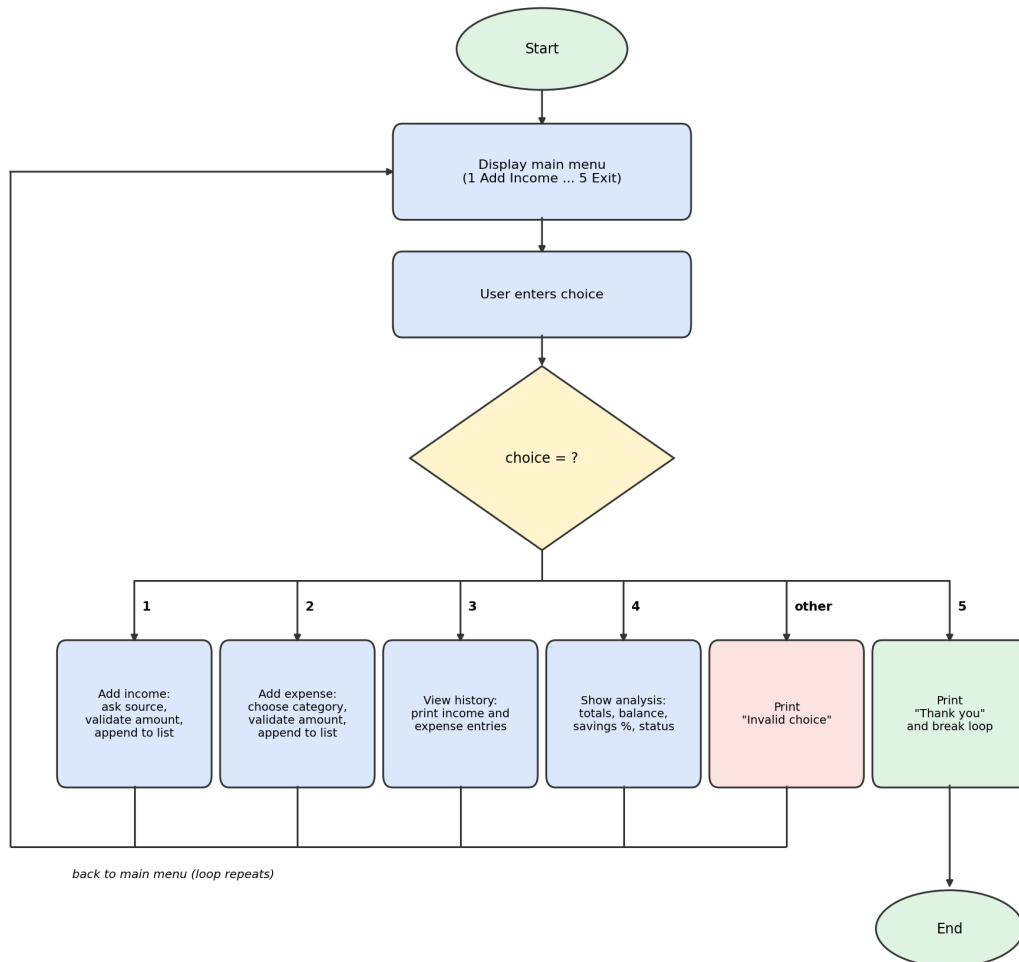


Figure 3: Workflow (process flow) diagram

6.3 Sequence Diagram

The sequence diagram shows the messages between the User, main.py, the functions and the data lists. Part A shows Add Income, including an invalid amount that is rejected before a valid one is accepted. Part B shows Show Analysis.

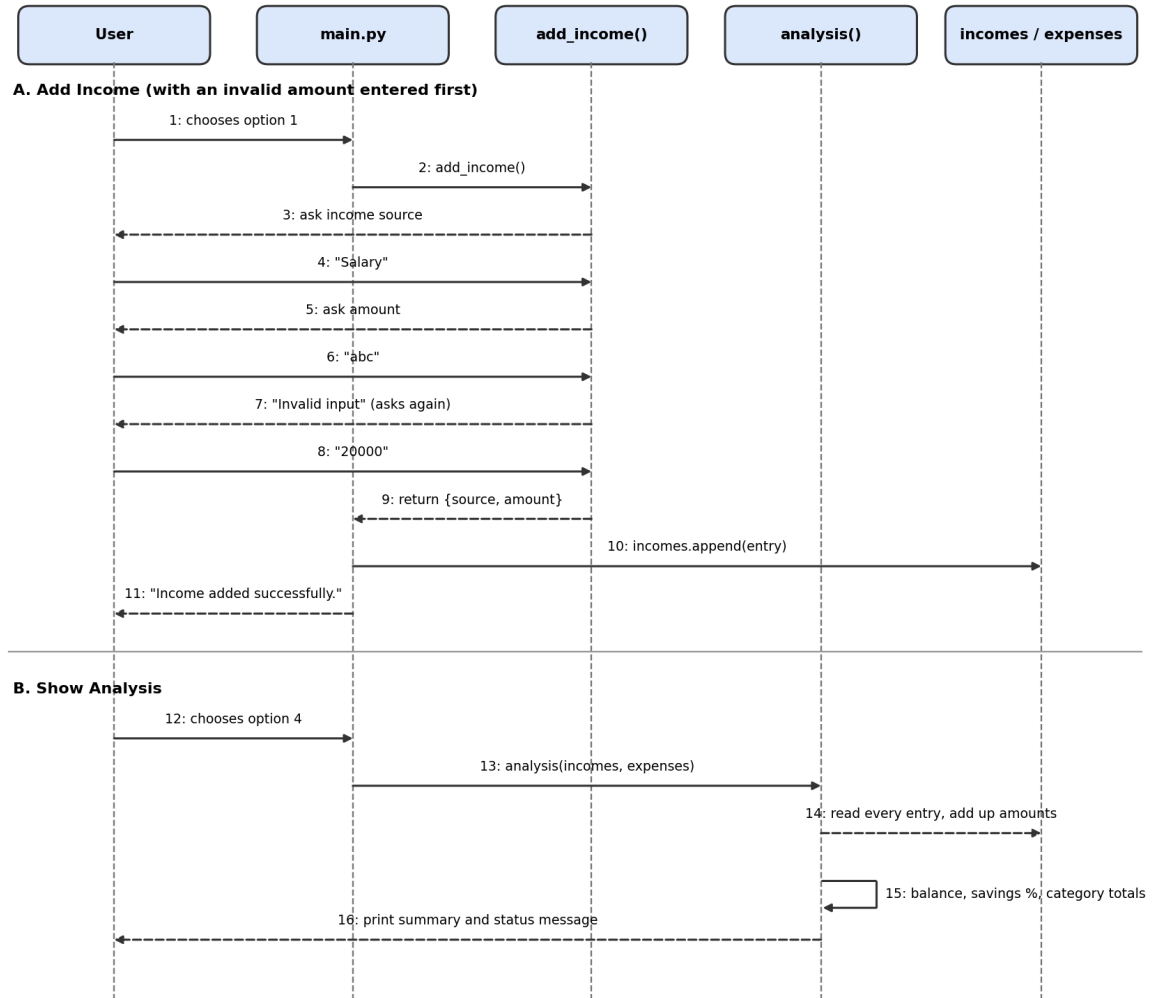


Figure 4: Sequence diagram for Add Income and Show Analysis

6.4 Class / Component Diagram

The project is built with functions and modules, so there are no classes. The component diagram therefore shows the modules, the functions they contain and how main.py uses them. Entries are stored as dictionaries.

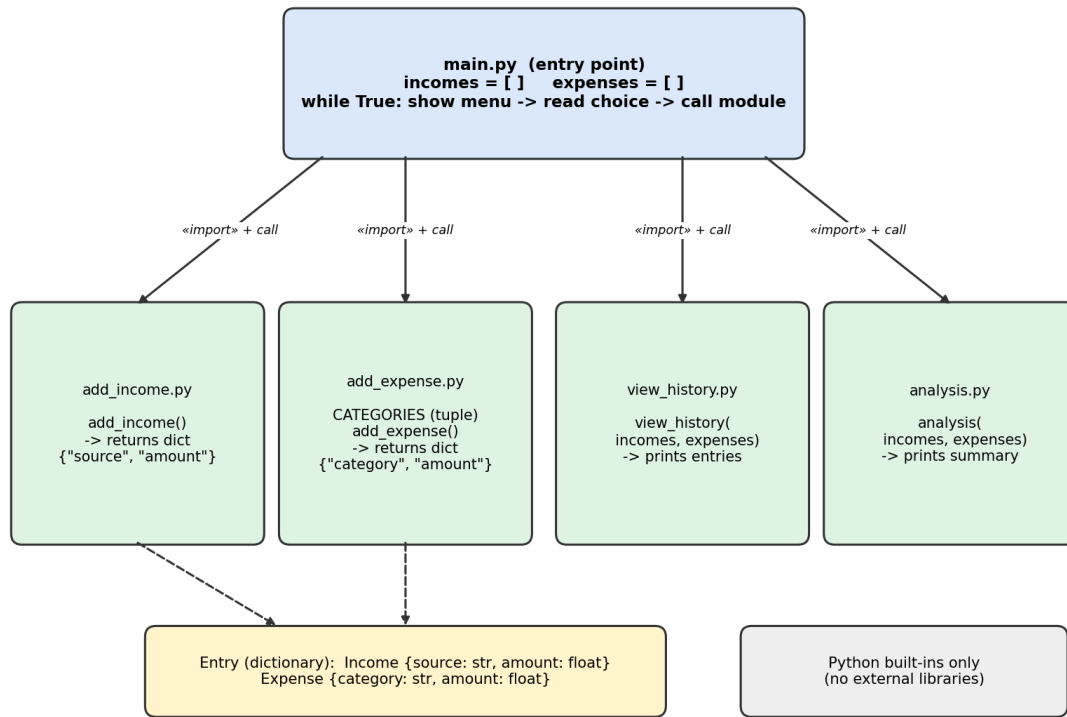


Figure 5: Component diagram

6.5 ER Diagram and Storage Design

The system does not use a database or files, so an ER diagram is not applicable. The data is stored only in memory while the program is running, in two lists of dictionaries:

Data structure	Type	Contents of each entry	Example
incomes	list of dict	source (str), amount (float)	<code>{"source": "Salary", "amount": 20000.0}</code>
expenses	list of dict	category (str), amount (float)	<code>{"category": "Food", "amount": 4000.0}</code>
CATEGORIES	tuple of str	Fixed list of six category names	<code>("Food", "Travel", "Shopping", "Bills", "Education", "Other")</code>
category_totals	dict (created inside analysis)	category name -> total spent	<code>{"Food": 4000.0, "Travel": 3500.0}</code>

7. Design Decisions and Rationale

Decision	Rationale
Command-line interface	Keeps the focus on the Python Essentials concepts, runs anywhere Python is installed and needs no extra libraries.
One file (module) per function	Makes the code modular and easy to maintain, and follows the course topic on user-defined modules. Each feature can be read and tested on its own.
Lists of dictionaries for entries	A dictionary keeps related values together under clear names (source or category, and amount), and a list keeps the order of entry.
Tuple for categories	The category names are fixed and should not change while the program runs, so a tuple is the right structure.
Amount validated with a character-check loop	Every character is checked with a for loop and the membership operator, and float() is used only after the input is known to be valid. This uses only concepts taught in the course.
Data stored in memory only	Keeps the program simple and within the course scope. Persistent storage is listed as a future enhancement.
Whole-number approximate percentages	int() converts the percentage to a whole number and the display says "approx", which keeps the output short and easy to read.
Analysis guarded against empty data	The savings rate is shown only when income is greater than 0 and the category breakdown only when expenses exist, which avoids division by zero.

8. Implementation Details

The project is implemented in Python 3 in five source files kept in one folder.

File	Function	Purpose
main.py	(main program)	Imports the four functions, creates the lists incomes and expenses, and runs the menu loop using while, if, elif and else.
add_income.py	add_income()	Asks for the income source and amount, validates the amount and returns a dictionary.
add_expense.py	add_expense()	Shows the categories from a tuple, asks for the category number and amount, validates the amount and returns a dictionary.
view_history.py	view_history(incomes, expenses)	Loops through both lists with for and range() and prints every entry.
analysis.py	analysis(incomes, expenses)	Adds up the amounts, calculates balance, savings rate and category totals (using a dictionary) and prints the summary and status.

8.1 Amount validation

Amounts are read as text with input(). A for loop checks each character: digits are counted, dots are counted and any other character makes the input invalid. The input is accepted only if it has at least one digit and at most one dot. float() then converts it, and an amount of 0 is rejected. The same logic is used in add_income.py and add_expense.py. Full source of add_income.py:

```
# add_income.py - function to add income

def add_income():
    source = input("Enter income source (e.g. Salary, Pocket Money): ")
    if source == "":
        source = "Other"

    # Keep asking until the user enters a valid positive number
    while True:
        text = input("Enter your income: ₹")
        valid = True
        dots = 0
        digits = 0

        # Check every character of the input
        for ch in text:
            if ch == ".":
                dots = dots + 1
            elif ch in "0123456789":
                digits = digits + 1
            else:
                valid = False

        if dots > 1 or digits == 0:
            valid = False

        if valid == False:
            print("Invalid input! Enter a positive number using digits only.")
            continue

        amount = float(text)    # type conversion: str -> float

        if amount == 0:
```

```

        print("Amount must be greater than 0.")
        continue

    break

# Store the entry as a dictionary
entry = {"source": source, "amount": amount}
return entry

```

8.2 Main program

main.py imports the functions, creates the two lists and repeats the menu until the user chooses Exit. Full source:

```

# main.py - Personal Expense Management System
# Run this file to start the program: python main.py

from add_income import add_income
from add_expense import add_expense
from view_history import view_history
from analysis import analysis

incomes = [] # list of income entries (dictionaries)
expenses = [] # list of expense entries (dictionaries)

while True:
    print("\n===== Personal Expense Management System =====")
    print("1. Add Income")
    print("2. Add Expense")
    print("3. View History")
    print("4. Show Analysis")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        incomes.append(add_income())
        print("Income added successfully.")

    elif choice == "2":
        expenses.append(add_expense())
        print("Expense added successfully.")

    elif choice == "3":
        view_history(incomes, expenses)

    elif choice == "4":
        analysis(incomes, expenses)

    elif choice == "5":
        print("Thank you for using the system!")
        break

    else:
        print("Invalid choice! Please try again.")

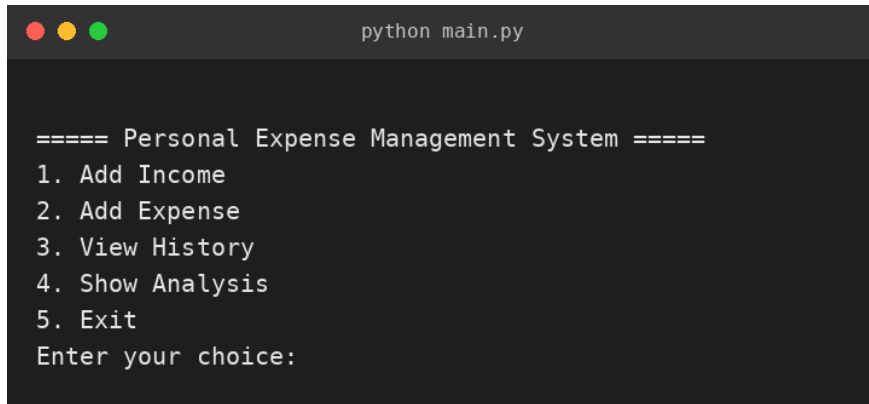
```

8.3 Analysis logic

analysis() loops through the incomes and expenses lists and adds up the amounts. The balance is total income minus total expense. The savings rate is balance divided by total income multiplied by 100 and converted with int(). For the category breakdown, a dictionary named category_totals is built: if the category is already a key its amount is added, otherwise a new key is created. The percentage of each category is its total divided by total expense multiplied by 100. Finally an if, elif and else chain prints the status message.

9. Screenshots / Results

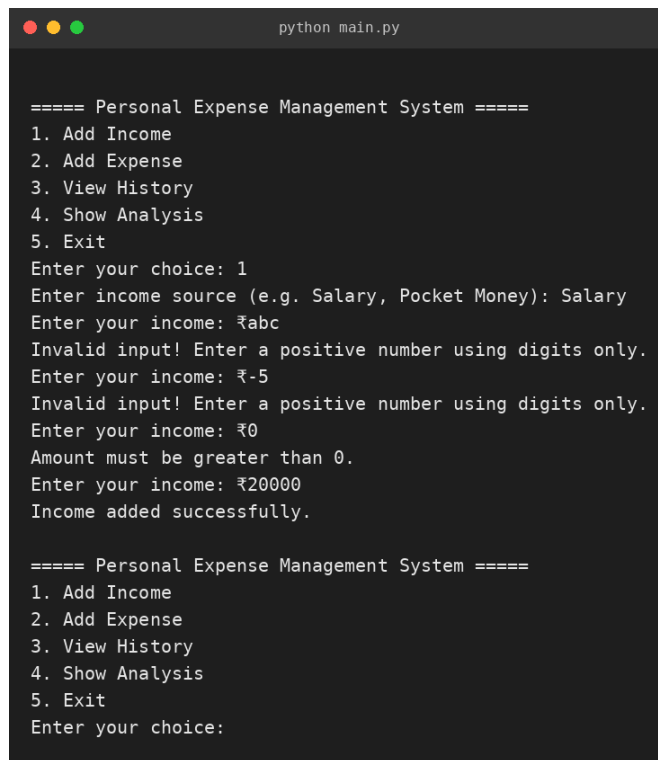
The screenshots below show the program running. They show the output produced by the program for the example data used in testing.



```
python main.py

===== Personal Expense Management System =====
1. Add Income
2. Add Expense
3. View History
4. Show Analysis
5. Exit
Enter your choice:
```

Figure 6: Main menu



```
python main.py

===== Personal Expense Management System =====
1. Add Income
2. Add Expense
3. View History
4. Show Analysis
5. Exit
Enter your choice: 1
Enter income source (e.g. Salary, Pocket Money): Salary
Enter your income: ₹abc
Invalid input! Enter a positive number using digits only.
Enter your income: ₹-5
Invalid input! Enter a positive number using digits only.
Enter your income: ₹0
Amount must be greater than 0.
Enter your income: ₹20000
Income added successfully.

===== Personal Expense Management System =====
1. Add Income
2. Add Expense
3. View History
4. Show Analysis
5. Exit
Enter your choice:
```

Figure 7: Invalid amounts (abc, -5, 0) rejected before a valid amount is accepted

```
python main.py

===== Personal Expense Management System =====
1. Add Income
2. Add Expense
3. View History
4. Show Analysis
5. Exit
Enter your choice: 1
Enter income source (e.g. Salary, Pocket Money): Salary
Enter your income: ₹20000
Income added successfully.

===== Personal Expense Management System =====
1. Add Income
2. Add Expense
3. View History
4. Show Analysis
5. Exit
Enter your choice: 2

Select a category:
1 . Food
2 . Travel
3 . Shopping
4 . Bills
5 . Education
6 . Other
Enter category number: 1
Enter your expense: ₹4000
Expense added successfully.

===== Personal Expense Management System =====
1. Add Income
2. Add Expense
3. View History
4. Show Analysis
5. Exit
Enter your choice:
```

Figure 8: Adding an income and an expense

```
python main.py

===== Personal Expense Management System =====
1. Add Income
2. Add Expense
3. View History
4. Show Analysis
5. Exit
Enter your choice: 3

----- Transaction History -----

Income:
1 . Salary      : ₹ 20000.0
2 . Pocket Money : ₹ 2000.0

Expenses:
1 . Food        : ₹ 4000.0
2 . Travel      : ₹ 3500.0

===== Personal Expense Management System =====
1. Add Income
2. Add Expense
3. View History
4. Show Analysis
5. Exit
Enter your choice:
```

Figure 9: Transaction history

```
python main.py

===== Personal Expense Management System =====
1. Add Income
2. Add Expense
3. View History
4. Show Analysis
5. Exit
Enter your choice: 4

----- Finance Analysis -----
Total Income  : ₹ 22000.0
Total Expense : ₹ 7500.0
Balance       : ₹ 14500.0
Savings Rate  : approx 65 %

Expense by category:
Food    : ₹ 4000.0 (approx 53 %)
Travel  : ₹ 3500.0 (approx 46 %)

Status: You have saved money.

===== Personal Expense Management System =====
1. Add Income
2. Add Expense
3. View History
4. Show Analysis
5. Exit
Enter your choice:
```

Figure 10: Finance analysis with category breakdown and status message

Result. For an income of ₹22000 and expenses of ₹4000 (Food) and ₹3500 (Travel), the analysis shows total expense ₹7500, balance ₹14500, and the message that the user has saved money.

10. Testing Approach

The program was tested manually with validation tests. Each test gives a specific input and compares the output of the program with the expected result. The tests cover invalid menu choices, invalid amounts, default values, empty data and the three status messages. All 15 tests were run and passed.

No .	Test	Input / steps	Expected result	Result
1	Invalid menu choice	Enter 9 at the menu	"Invalid choice! Please try again."	Pass
2	Letters as amount	Add income, amount: abc	Invalid input message, asks again	Pass
3	Negative amount	Add income, amount: -5	Invalid input message, asks again	Pass
4	Zero amount	Add income, amount: 0	"Amount must be greater than 0."	Pass
5	Two decimal points	Add income, amount: 1.2.3	Invalid input message, asks again	Pass
6	Empty amount	Add income, press Enter	Invalid input message, asks again	Pass
7	Digits with letters	Add income, amount: 10rs	Invalid input message, asks again	Pass
8	Decimal amount	Add income, amount: 250.50	Accepted, income added successfully	Pass
9	Empty income source	Press Enter for source, amount 100	Saved with source "Other"	Pass
10	Wrong category number	Add expense, category 9, amount 100	Saved with category "Other"	Pass
11	History with no data	Choose option 3 at start	"No income added yet." and "No expenses added yet."	Pass
12	Analysis with no data	Choose option 4 at start	Totals are 0 and the equal status message	Pass
13	Overspending	Income 1000, expense 1500, analysis	Balance -500.0 and "spent more than your income"	Pass
14	Income equals expense	Income 1000, expense 1000, analysis	Balance 0.0 and "income and expense are equal"	Pass
15	Same category twice	Food 100 and Food 50, analysis	Food shown once with total 150.0	Pass

11. Challenges Faced

- **Working within the course syllabus.** Some convenient Python features such as try/except, enumerate(), f-strings and round() are not part of the course. The program had to be redesigned to use only taught concepts.
- **Validating input without try/except.** A simple float(input()) crashes on wrong input. The solution was to check each character with a for loop and the membership operator before converting the value with float().
- **Neat output without formatting methods.** Without format specifiers, decimals and alignment were harder to control. Plain print() with several values, the escape sequences \n and \t, and int() for percentages were used instead.

- **Avoiding errors on empty data.** Calculating percentages when there is no income or no expenses would divide by zero. Conditions were added so these values are calculated only when data exists.
- **Choosing a category without `isdigit()`.** The category number is compared with `str(i + 1)` inside a loop, so any wrong value simply falls back to "Other".
- **Organising the code into modules.** The functions had to be split into separate files and imported correctly, which requires all files to be kept in the same folder.
- **Meeting the project size requirement.** The project needed several meaningful files and documentation, so the program was divided by function and a README, statement and this report were written.

12. Learnings and Key Takeaways

- Splitting a program into modules makes it easier to read, test and change.
- Dictionaries and lists together are a simple and powerful way to store structured data.
- Input should never be trusted: every value entered by the user must be validated.
- Small details such as division by zero and empty data must be handled to make a program reliable.
- Writing test cases with expected results helps to find mistakes early.
- Good documentation (README, statement and report) is as important as the code itself.
- Version control with Git keeps a history of changes and makes the project easy to share.

13. Future Enhancements

- Save entries to a file so that data is not lost when the program closes.
- Add options to edit and delete entries (complete CRUD operations).
- Add a monthly budget limit with a warning when spending crosses it.
- Search and filter entries by category.
- Add dates to entries and generate monthly reports.
- Add a graphical or web interface.

14. References

- VITyarthi, Python Essentials course material, Modules 1 to 10.
- VITyarthi, Build Your Own Project - General Project Instructions and Submission Guidelines.
- Python Software Foundation, Python 3 Documentation, <https://docs.python.org/3/>
- Python Software Foundation, The Python Tutorial, <https://docs.python.org/3/tutorial/>
- GitHub